



MODULE 8

OBJECT ORIENTED ANALYSIS AND MODELING

Prepared by:

Dr. Melani L. Castillo

Faculty in Charge





OBJECTIVES



- Identify the steps in object oriented analysis and design
- Explain unified modeling language (UML)
- Discuss the various types of UML diagrams.



The Concept Of Object-Orientation



- **Object-orientation** is what is referred to as a programming paradigm. It is not a language itself but a set of concepts that is supported by many languages.
- In an object-oriented language, one large program will instead be split apart into self contained **objects**, almost like having several mini programs, each **object** representing a **different part** of the application.
- And each object contains its **own data and its own logic**, and they communicate between themselves.
- They represent things like employees, images, bank accounts, spaceships, asteroids, video segment, audio files, or whatever exists in your program.





Object-Oriented Analysis And Design (OOAD)



- It is a **structured method** for analyzing, designing a system by applying the object orientated concepts, and develop a set of graphical system models during the development life cycle of the software.
- The goal of the analysis phase is to *create a **model of the system regardless of constraints such as appropriate technology.***
- The design phase *refines the analysis model and applies the needed technology and other implementation constrains.* It focuses on describing the objects, their attributes, behavior, and interactions.





Object-Oriented Analysis



In the **object-oriented analysis**, we ...

- 1. **Elicit** *requirements*: Define what does the software need to do, and what's the problem the software trying to solve.
- 2. **Specify** *requirements*: Describe the requirements, usually, using use cases (and scenarios) or user stories.
- 3. **Conceptual** *model*: Identify the important objects, refine them, and define their relationships and behavior and draw them in a simple diagram





Object-Oriented Design



In the **object-oriented design**, we ...

1. **Describe the classes** and their relationships using class diagram.
2. **Describe the interaction** between the objects using sequence diagram.
3. Apply software design principles and design patterns.



System Modeling



- **System Modeling** is the process of **developing models** of the system, with each model representing a different perspective of that system.
- The most important aspect about a system model is that it leaves out detail; it is an **abstract representation** of the system.
- The models are usually based on graphical notation, which is almost always based on the notations in the Unified Modeling Language (UML).





Different Perspectives of Modeling



1. **External**, where you model the context or the **environment of the system**.
2. **Interaction**, where you model the **interaction between components of a system**, or between a system and other systems.
3. **Structural**, where you model the **organization of the system**, or the **structure of the data** being processed by the system.
4. **Behavioral**, where you model the **dynamic behavior of the system and how it respond to events**.





Unified Modeling Language (UML)



- **Unified Modeling Language (UML)** is a general purpose modelling language.
- The main aim of UML is to define a standard way to **visualize the way a system has been designed**.
- UML is **not a programming language**; it is rather a visual language. We use UML diagrams to portray the **behavior and structure** of a system.





Object Oriented Concepts Used in UML



1. **Class** – A class defines the blue print i.e. structure and functions of an object.
2. **Objects** – Objects help us to decompose large systems and help us to modularize our system. An object is the fundamental unit (building block) of a system which is used to depict an entity.
3. **Inheritance** – Inheritance is a mechanism by which child classes inherit the properties of their parent classes.





Object Oriented Concepts Used in UML



4. **Abstraction** – Mechanism by which implementation details are hidden from user.

5. **Encapsulation** – Binding data together and protecting it from the outer world is referred to as encapsulation.

6. **Polymorphism** – Mechanism by which functions or entities are able to exist in different forms.



The Origin of UML



- 1. **James Rumbaugh 1991 - Object Modeling Technique OMT** - was best for analysis and data-intensive information systems.
- 2. **Grady Booch 1994** - had worked extensively with the **Ada** language, and had been a major player in the development of Object Oriented techniques for the language.
- 3. **Ivar Jacobson 1992 - OOSE (Object-Oriented Software Engineering** - featured a model known as Use Cases. Use Cases are a powerful technique for understanding the behaviour of an entire system.





The Origin of UML

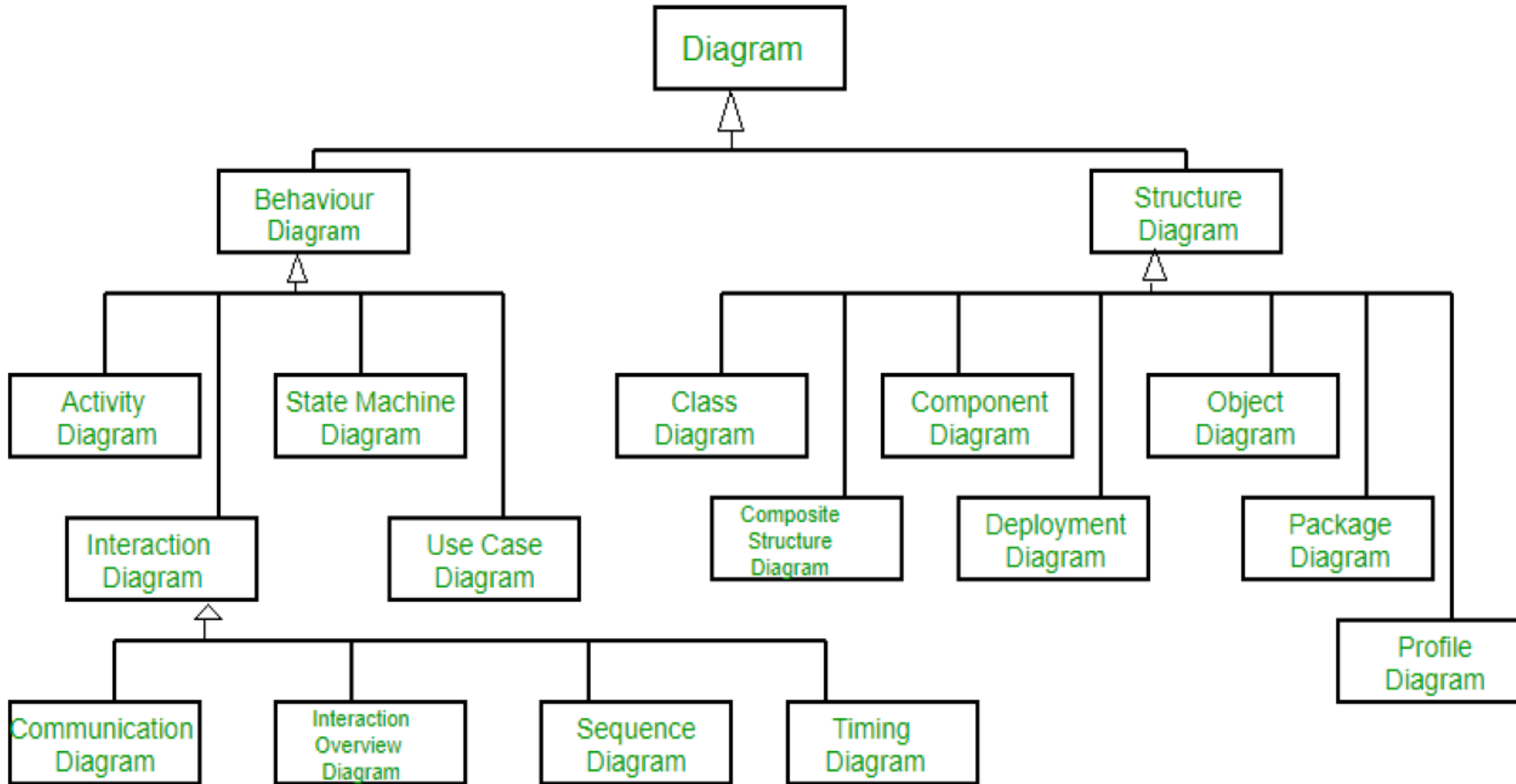


- **In 1994**, Jim Rumbaugh, the creator of OMT, stunned the software world when he left General Electric and joined Grady Booch at Rational Corp. The aim of the partnership was to merge their ideas into a single, unified method named "Unified Method".
- **By 1995**, the creator of OOSE, Ivar Jacobson, had also joined Rational, and his ideas were fed into the new Unified Method - now called the Unified Modelling Language¹. The team of Rumbaugh, Booch and Jacobson are affectionately known as the "**Three Amigos**".
- The **Object Management Group (OMG)** adopted Unified Modelling Language as a standard in 1997. It has been managed by OMG ever since International Organization for Standardization (ISO) published UML as an approved standard in 2005.





Diagrams in UML





Diagrams in UML



Most diagrams that are **commonly used** for UML are:

- **Use case diagram**: It shows the interaction between a system and its environment (users or systems) within a particular situation.
- **Class diagram**: It shows the different objects, their relationship, their behaviors, and attributes.
- **Activity diagram**: It shows the flow of the data between the processes in the system

Sequence diagram: It shows the interactions between the different objects in the system, and between actors and the objects in a system.

- **State machine diagram**: It shows how the system responds to external and internal events.



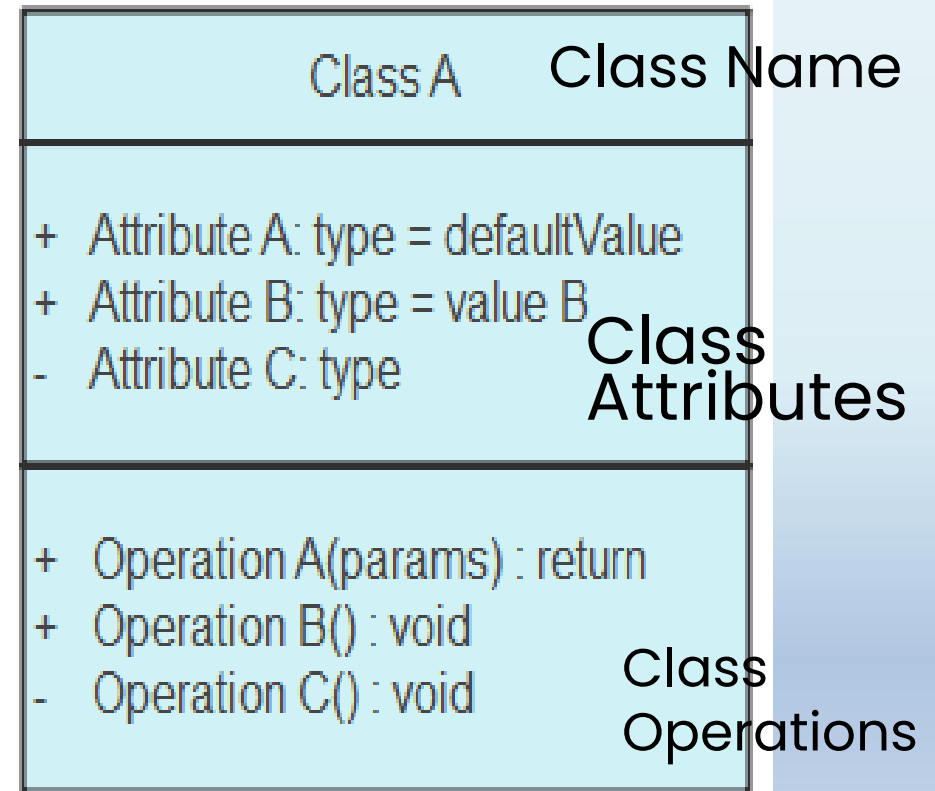


Class diagram



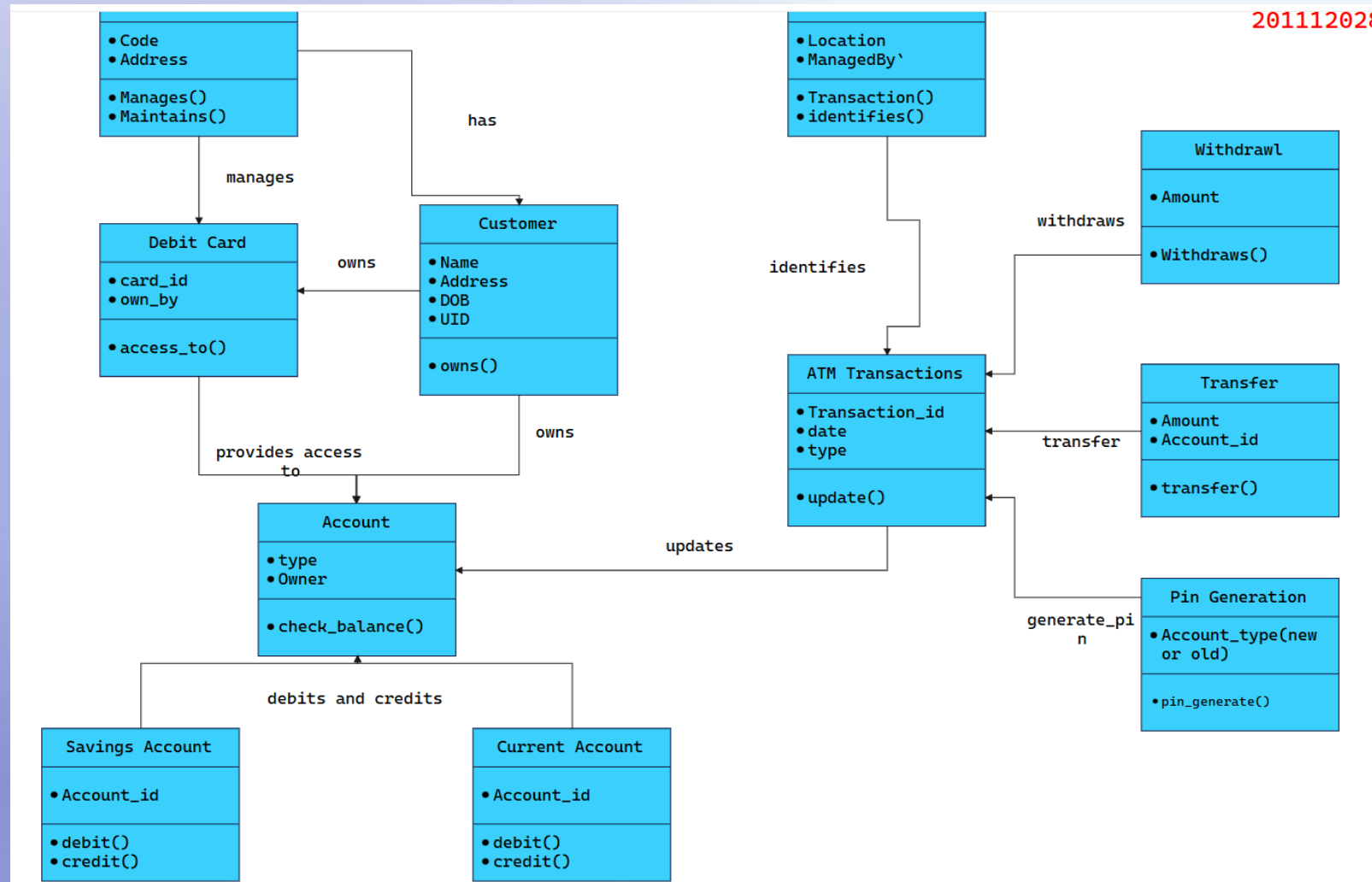
The **class diagram** is a central modeling technique that runs through nearly all object-oriented methods. This diagram describes the types of objects in the system and various **kinds of static relationships which exist between them.**

Three major parts of a class diagram





Class Diagram Example





What is Component Diagram?

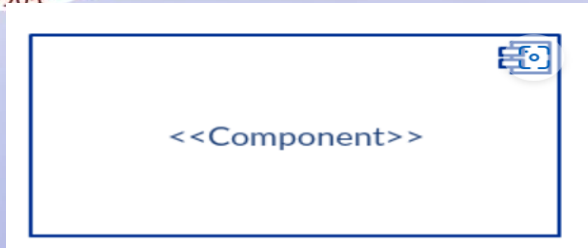
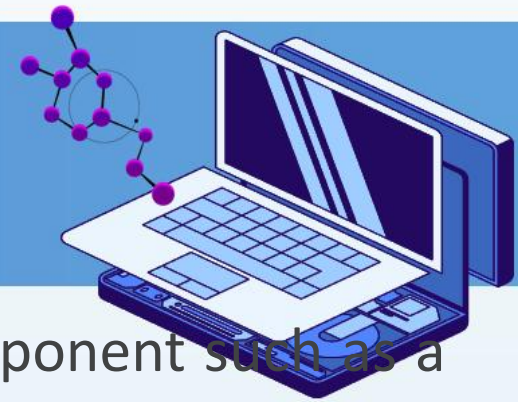


- In the Unified Modeling Language, a component diagram depicts **how components are wired together to form larger components or software systems.**
- It illustrates the architectures of the software components and the dependencies between them.
- Those software components including run-time components, executable components also the source code components.

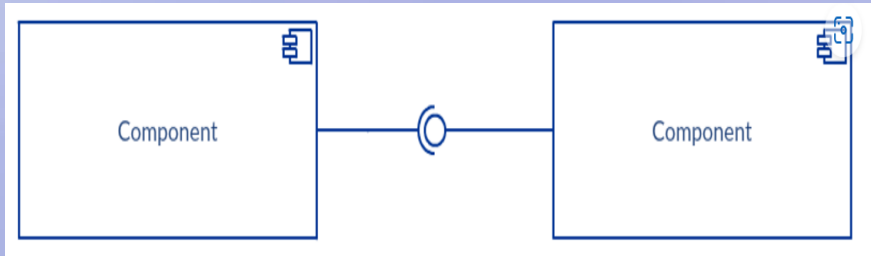




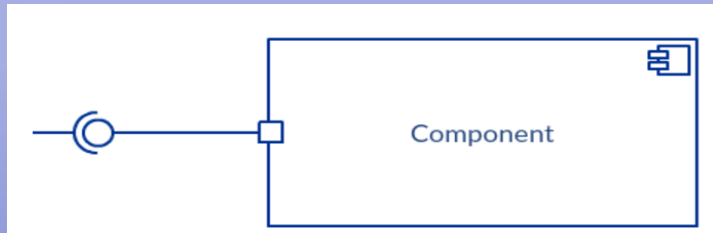
Component Diagram Symbols



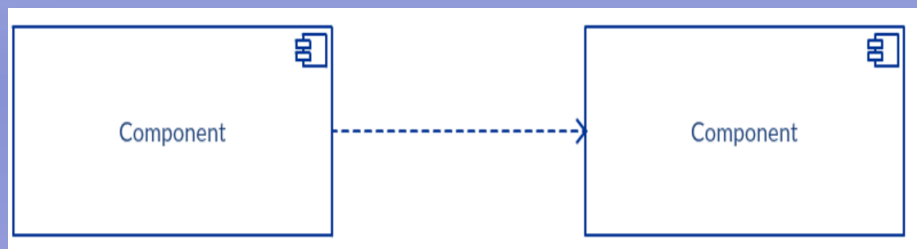
The components can be a software component such as a database or user interface; or a hardware component such as device; or a business unit such as supplier, etc.



Interfaces in component diagrams show how components are wired together and interact with each other.



Port is used when the component delegates the interfaces to an internal class.



Dependency show the relationship between two components.





How to Draw a Component Diagram

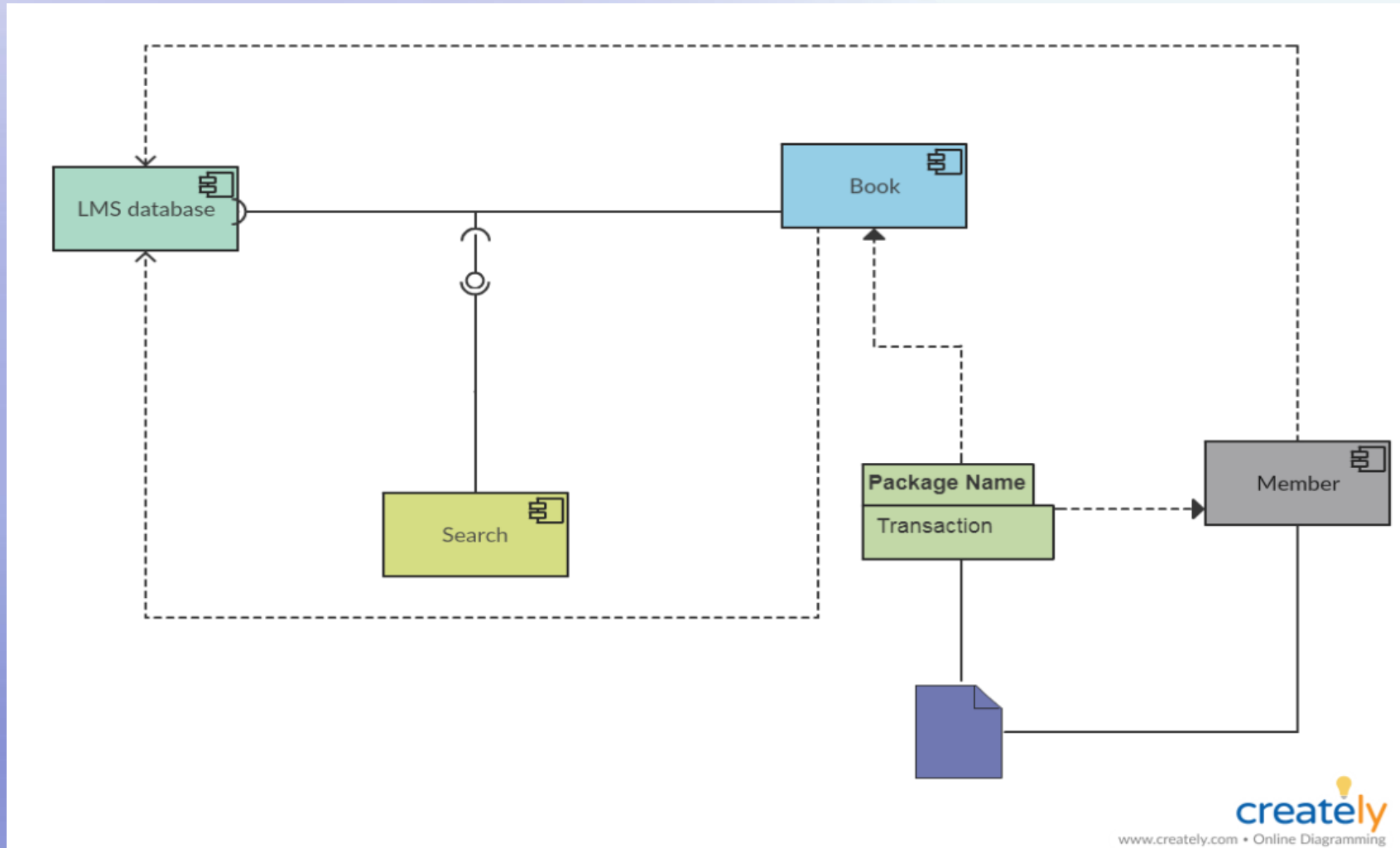


- 1:** Figure out the purpose of the diagram and identify the artifacts such as the files, documents etc. in your system or application that you need to represent in your diagram.
- 2:** As you figure out the relationships between the elements you identified earlier, create a mental layout of your component diagram
- 3:** As you draw the diagram, add components first, grouping them within other components as you see fit
- 4:** Next step is to add other elements such as interfaces, classes, objects, dependencies etc. to your component diagram and complete it.
- 5:** You can attach notes on different parts of your component diagram to clarify certain details to others.





Component Diagram Example





What is an Object Diagram?



- An **object diagram** is a graph of instances, including **objects and data values**.
- A static object diagram is an instance of a class diagram; it shows a **snapshot of the detailed state of a system at a point in time**.
- The difference is that a **class diagram** represents an abstract model consisting of classes and their relationships.
- However, an object diagram represents an instance at a particular moment, which is concrete in nature.
- The use of object diagrams is fairly limited, namely to show examples of data structure

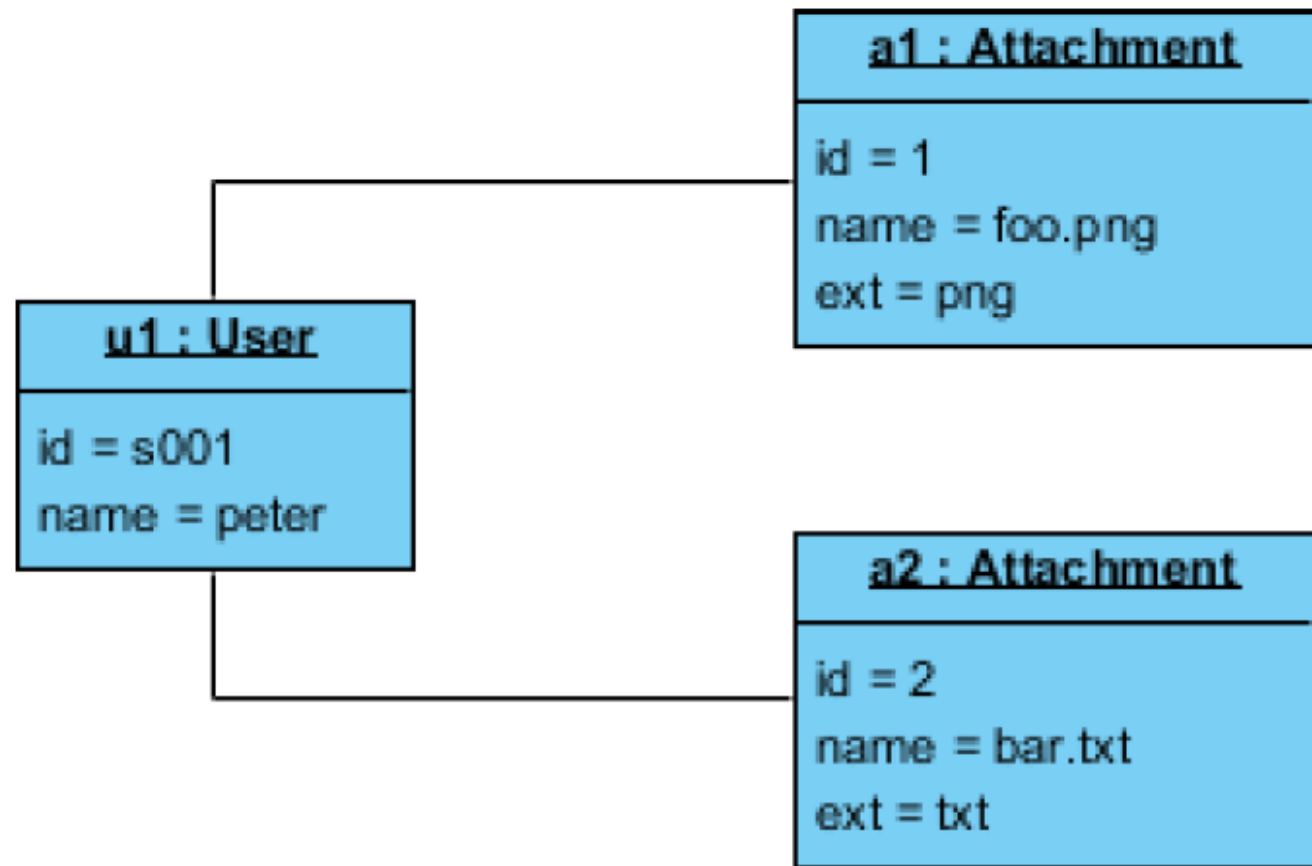




Object Diagram Example



The following Object Diagram example shows you how the object instances of User and Attachment class "look like" at the moment Peter (i.e. the user) is trying to upload two attachments. So there are two Instance Specification for the two attachment objects to be uploaded.

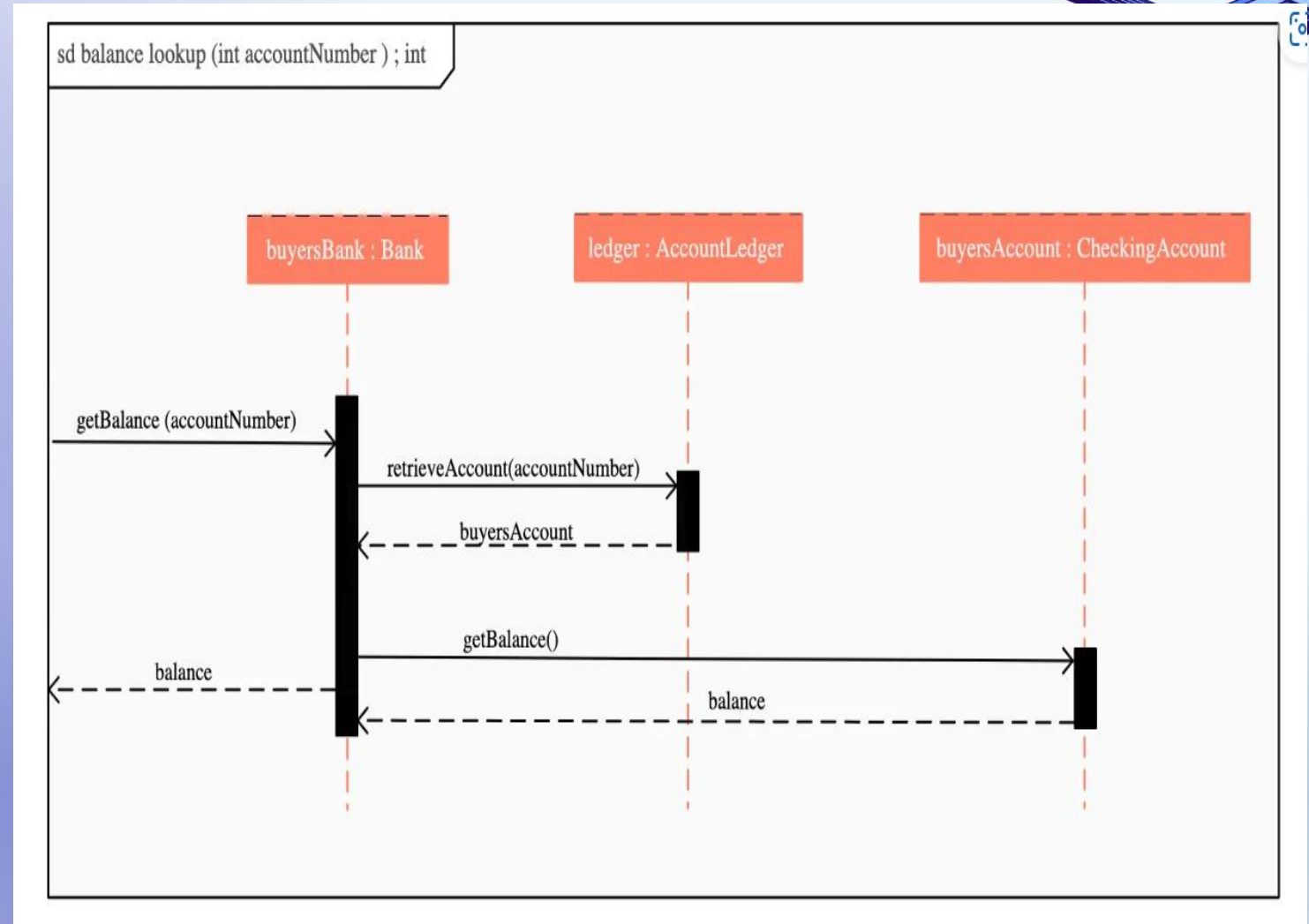




Sequence diagram



- It shows the interactions between the different objects in the system, and between actors and the objects in a system.





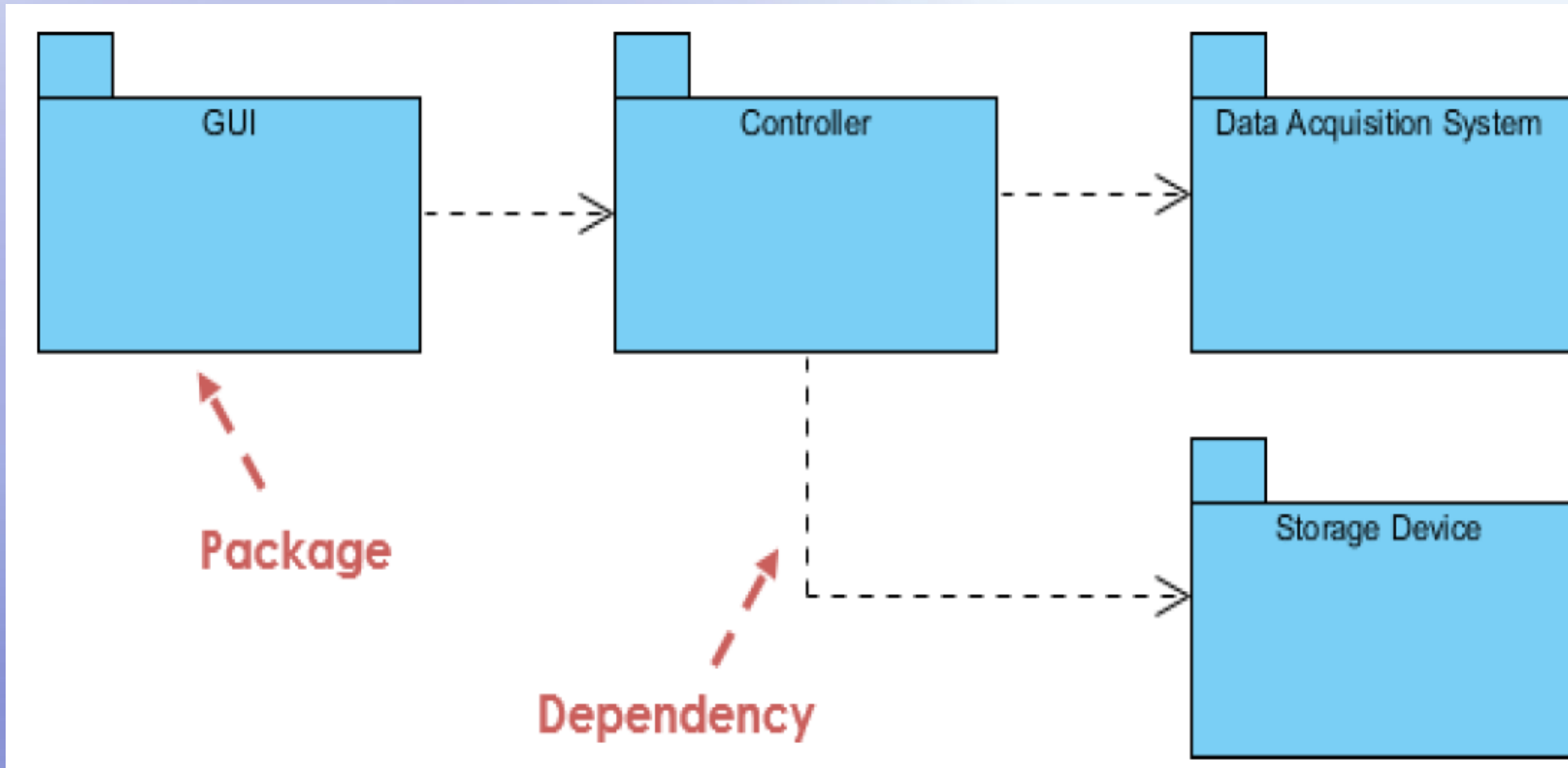
What is a Package Diagram?



- **Package diagram** is UML structure diagram which shows **packages and dependencies between the packages**.
- This model diagrams allow to show different views of a system, for example, as multi-layered (aka multi-tiered) application - multi-layered application model.



What is a Package Diagram?





What is a Deployment Diagram?

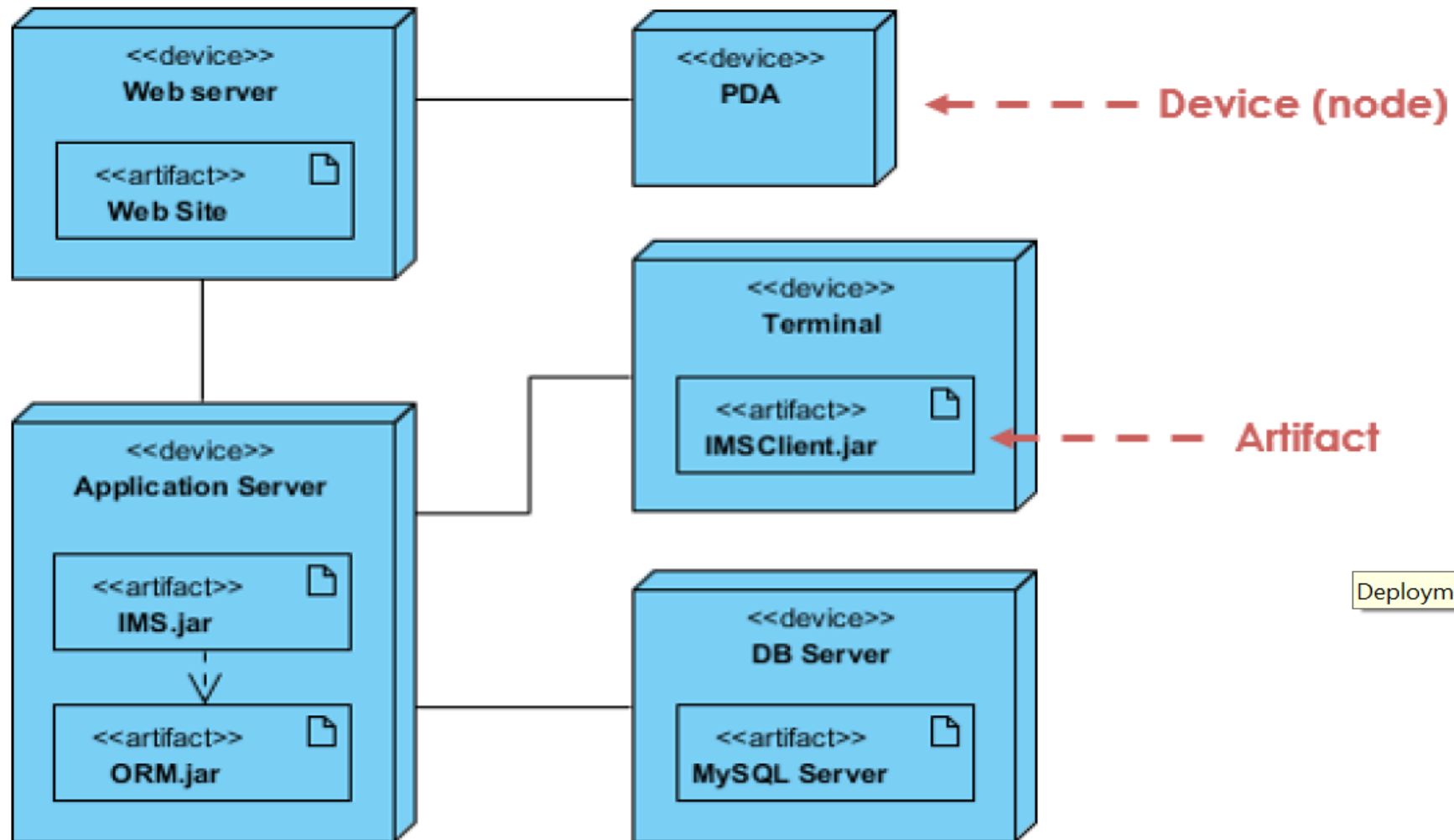


- The **Deployment Diagram** helps to model the **physical aspect** of an Object-Oriented software system. It is a structure diagram which shows **architecture of the system as deployment (distribution) of software artifacts** to deployment targets.
- Artifacts represent concrete elements in the physical world that are the result of a development process. It models the run-time configuration in a static view and visualizes the distribution of artifacts in an application.
- In most cases, it involves modeling the hardware configurations together with the software components that lived on.





Deployment Diagram Example



Deployment Diagram

- [UML Deployment Diagram](#)





End of Presentation



- Questions?



EXERCISES:



1. Create the following diagrams for a Student Information System (SIS) with the following modules:

- Faculty Module – allows faculty to enter student grades.
- Student Module – allows student to enroll their subjects.
- Registrar Module – use to generate student credentials.

- A. Class Diagram**
- B. Component Diagram**
- C. Sequence Diagram**
- D. Package Diagram**
- E. Deployment Diagram**





EXERCISES:



2. Create the following diagrams for a Library System with the following modules:

- Borrow/Return Module – use to record borrowed and returned books.
- Book Module – allows librarian to enter new books.
- Student Module – use to enter student library accounts

A. Class Diagram

B. Component Diagram

C. Sequence Diagram

D. Package Diagram

E. Deployment Diagram

